# Ensemble Learning

# Papers name

- Ensemble methods in machine learning,
  *Thomas G. Dietterich*, *Oregon state university*

- The boosting approach to machine learning: An overview,
  Robert E. Schapire, AT&T research

# Outline

- Introduction
- Ensemble of classifiers
- Why ensemble perform better
- Existing methods
- AdaBoost
- Summary

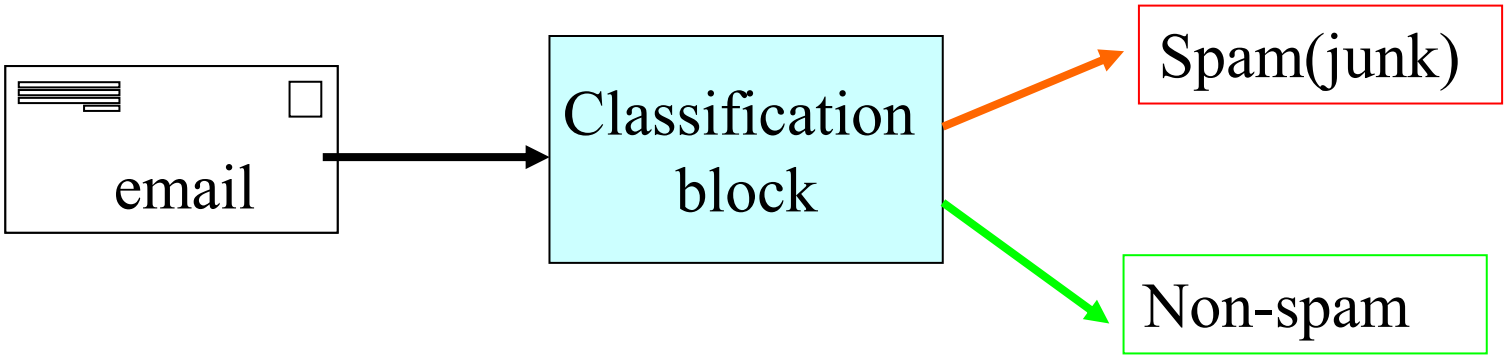
# כללי

- עד כה, למדנו פתרון בעיית סיווג המבוסס על היפוטזה בודדת בלבד (למשל, סיווג עפ"י עץ החלטה יחיד)

- בשיטת ה-Ensemble אנו משלבים מספר היפוטזות כדי לקבל החלטה

- דוגמא: ייצר 50 עצי החלטה שונים (איך???) והחלט ("סווג") לפי החלטת רוב העצים.

# Example: email filter



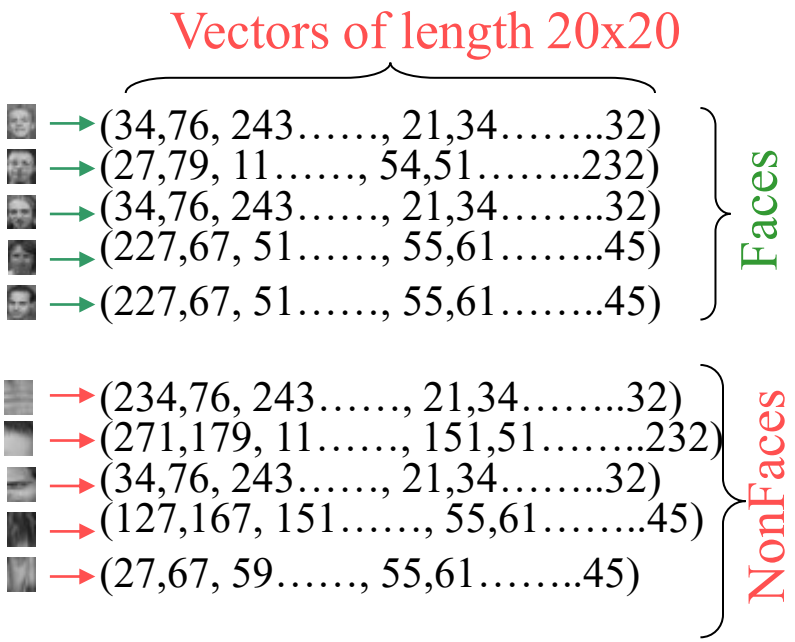

**Rules of thumb:**

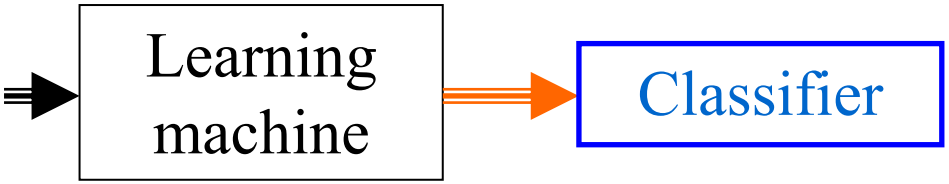
•If the phrase “buy now” occurs DECIDE ‘Spam’

# An observation

מציאת הרבה חוקים פשוטים ("כללי אצבע") לפתרון הבעיה הינה קלה בהרבה ממציאת חוק בודד ומאוד מדויק לפתרון בעיית החיזוי

# Learning algorithm:






# Learning algorithm:

- ***Given:***

  A set S of Training examples: $\{(x_1,y_1).....(x_m,y_m)\}$

  – Where:

  - $x_i$ is a vector of n features: $(x_{i1}, x_{i2}, x_{i3}, .....x_{in})$
  - $y_i$ values are represents a discrete set of classes (1,2,….K)

- ***Output (h):***

  Hypothesis (Classifier) about the true function y=f(x),

  i.e. given a new vector x, it predicts the corresponding y.

# Ensemble of classifiers

- **הגדרה:**

  – קבוצה של מסווגים (classifiers) שההחלטות שלהם משולבות בצורה כלשהיא ע"מ לסווג ווקטורים חדשים.

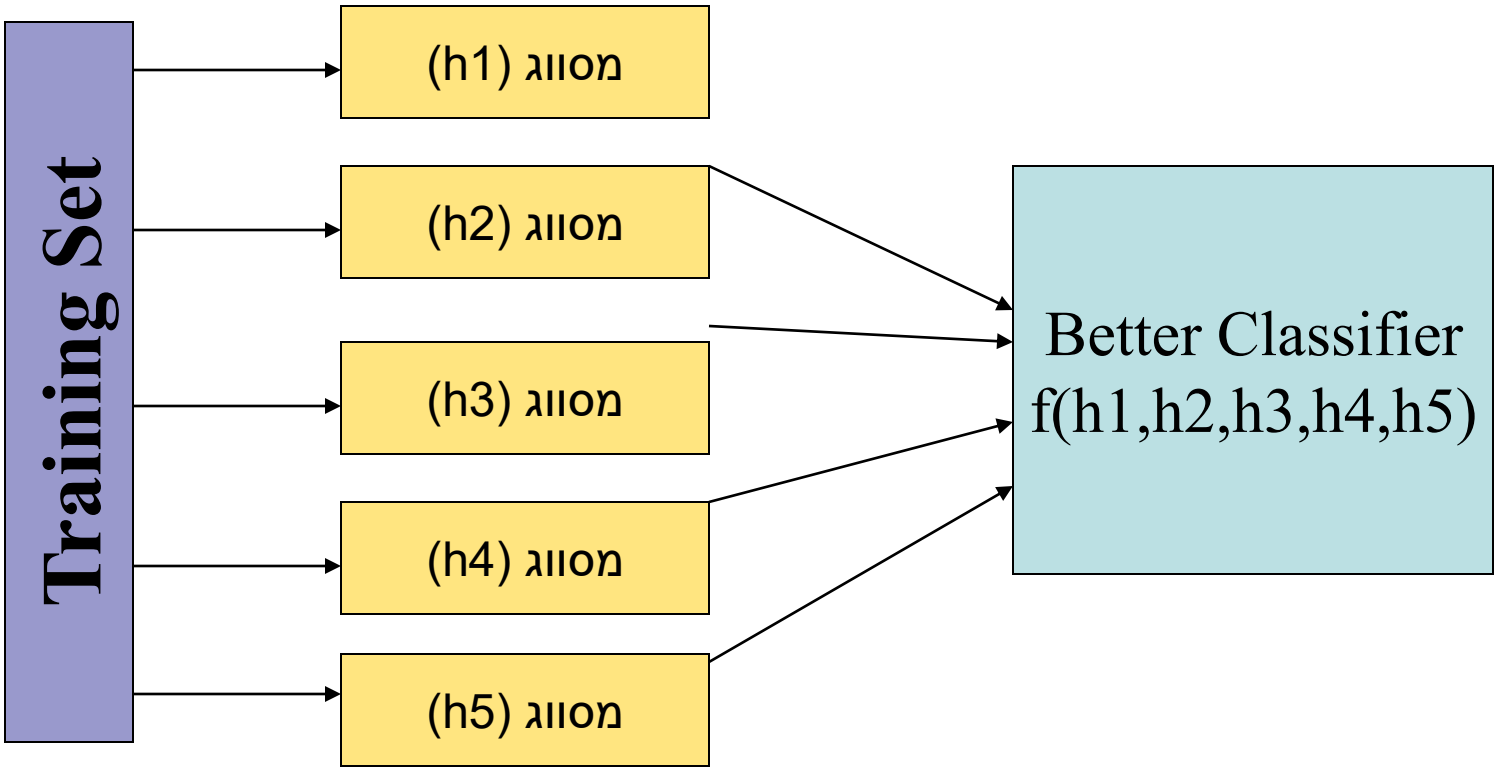
- **הרווח**

  – שקלול מסווגים הרבה יותר מדוייק מאשר שימוש בכל מסווג בצורה פרטנית.

# תכונות של המסווגים הבודדים

- **רמת דיוק**
  – צריכה להיות טובה יותר מאשר ניחוש אקראי ("הטלת מטבע")
- **"מגוונים"**
  – כלומר, הטעות של כל שני מסווגים בודדים על אותו מבחן תהיה שונה

# Schematic description



# דוגמא



$h_1$

$\varepsilon(h_1)=0.3$



$h_2$

$\varepsilon(h_2)=0.3$



$h_3$

$\varepsilon(h_3)=0.3$

**ונניח שאנו רוצים להכריע עפ"י "הצבעת רוב",**

כלומר עבור כל ווקטור חדש להריץ את שלושת המסווגים $h_1,h_2,h_3$ ולהכריע עפ"י רוב. כלומר אם לפחות שני מסווגים הכריעו סיווג $i$ – זו תהיה הכרעתנו.

# דוגמא

| $h_1$ | $h_2$ | $h_3$ |
|---|---|---|
| $\varepsilon(h_2)=0.3$ | $\varepsilon(h_2)=0.3$ | $\varepsilon(h_3)=0.3$ |

כדי שנסווג סיווג טעות בשיטת "הצבעת רוב"

- או **ששלושת המסווגים טועים**: $0.3\times0.3\times0.3$
- או **ששניים מתוך שלושת** המסווגים טועים:
  - למשל טעות של $h_1,h_2$ ו-$h_3$ צודק: $0.3\times0.3\times0.7$
  - יש לנו שלוש אפשרויות כאלו ולכן: $3\times0.3^2\times0.7$
- סה"כ הטעות:

$$\textbf{Error} = \mathbf{0.3^3} + \mathbf{3\times0.3^2\times0.7} = \mathbf{0.216 < 0.3}$$

$$Perror = \sum_{i=2}^{3} \binom{3}{i} p^i (1-p)^{3-i}$$

# Majority Vote

- באופן כללי, אם נניח שיש לנו $L$ מסווגים שונים, ולכל מסווג ההסתברות לטעות היא $p$ *אזי* ההסתברות לטעות הכוללת הינה:

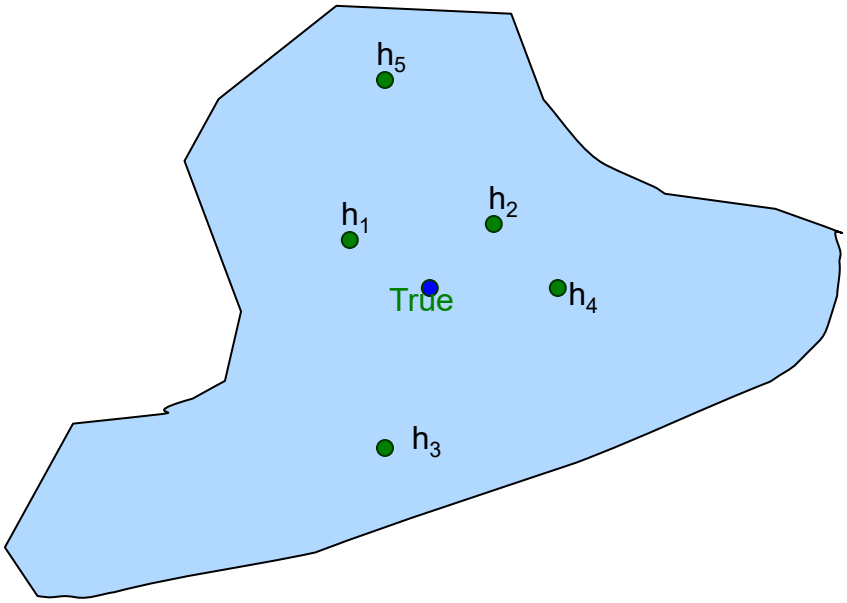
$$Perror = \sum_{i=\frac{L}{2}}^{L} \binom{L}{i} p^i (1-p)^{L-i}$$

# למה "איחוד מסווגים" הוא טוב יותר

- סטטיסטית
- חישובית
- ייצוגית

# סטטיסטית

- כאשר יש לנו כמות נתונים קטנה, אזי ניתן למצוא סוגים שונים של היפוטזות שלכל אחת מהן יש טעות מסויימת.
- השקלול עשוי למזער את ה"סיכון" לטעות



# חישובית

- באלגוריתמים מסויימים אנו מובטחים למציאת מינימום מקומי (ולא גלובלי)
- מציאת המינימום הגלובלי עשויה להיות "יקרה" חישובית עבור כמות נתונים גדולה.
- שקלול אלגוריתמים שבכל אחד מהם מובטח לנו מינימום מקומי עשוי לספק קירוב טוב יותר.

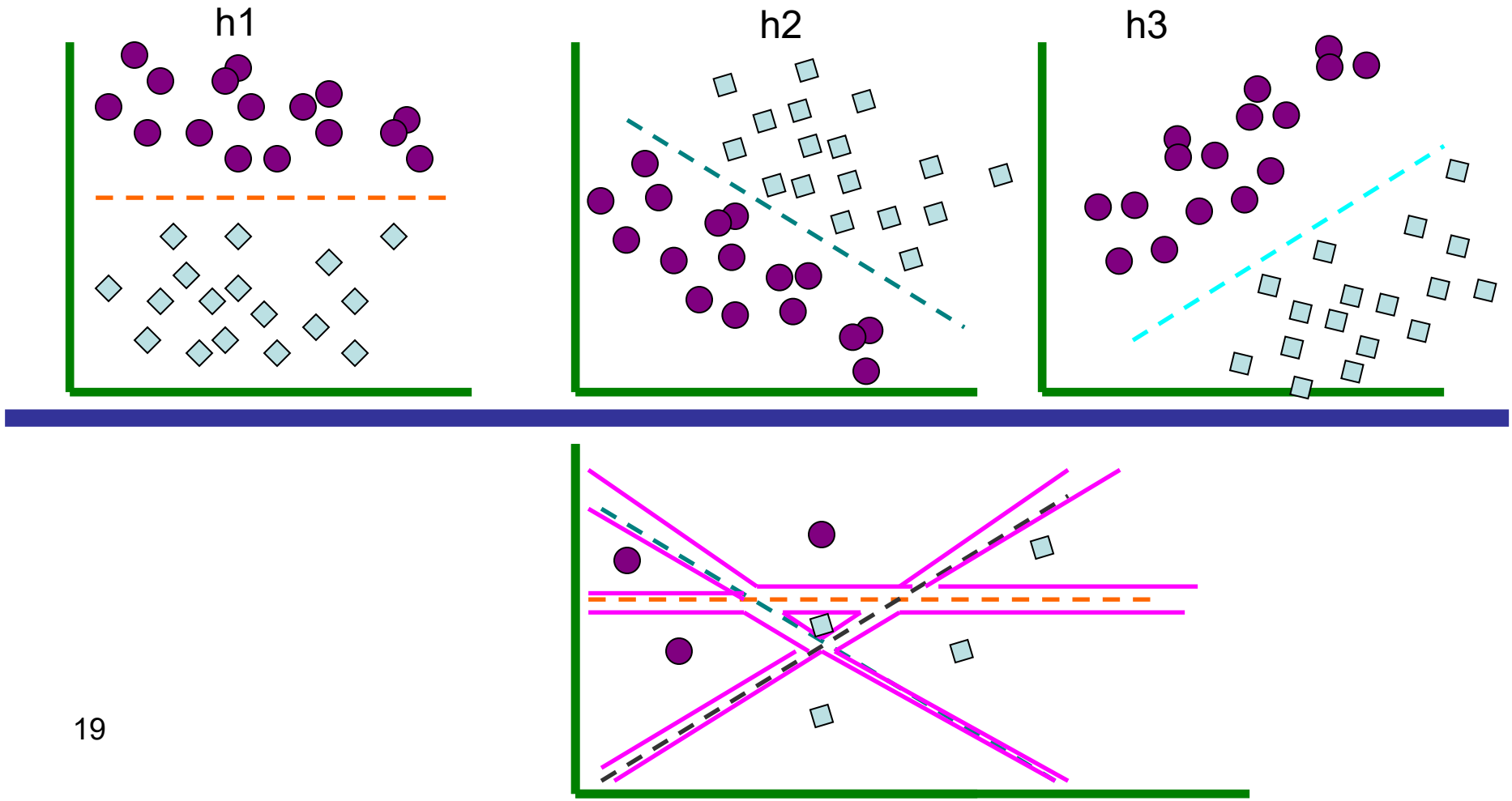
# ייצוגית

- לעיתים רבות, אין לנו את היכולת לייצג במרחב ההיפוטזות אותו אנו סורקים את ההיפוטזה ה"אמיתית" שפותרת את בעיית הלמידה שלנו.

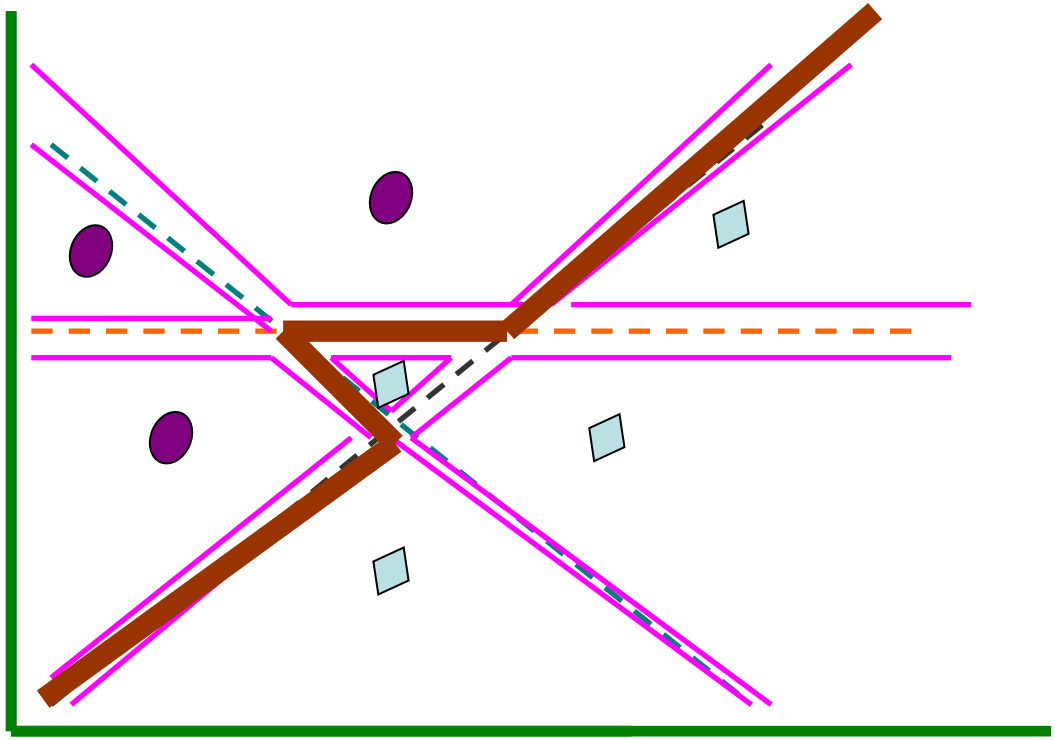
- שקלול היפוטזות יאפשר לנו ייצוג שלא ניתן להגיע אליו במרחב ההיפוטזות הבסיסי אותו אנו סורקים.

# ייצוגית

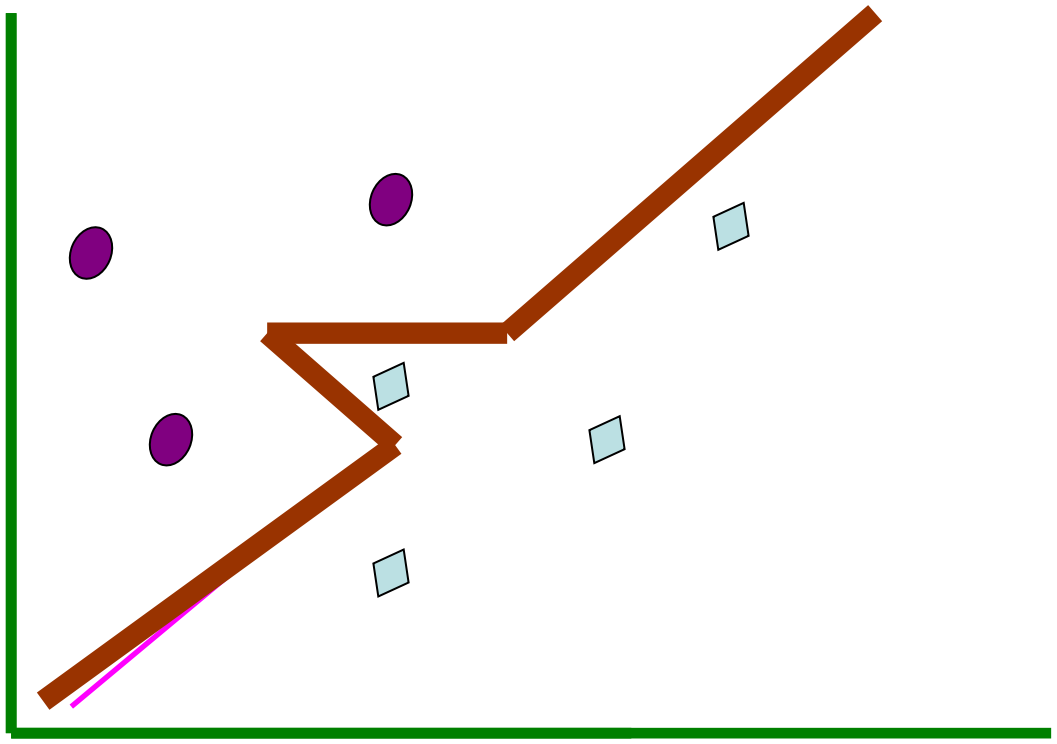
- Suppose we have in H only Linear hyperplanes



# ייצוגית



# ייצוגית

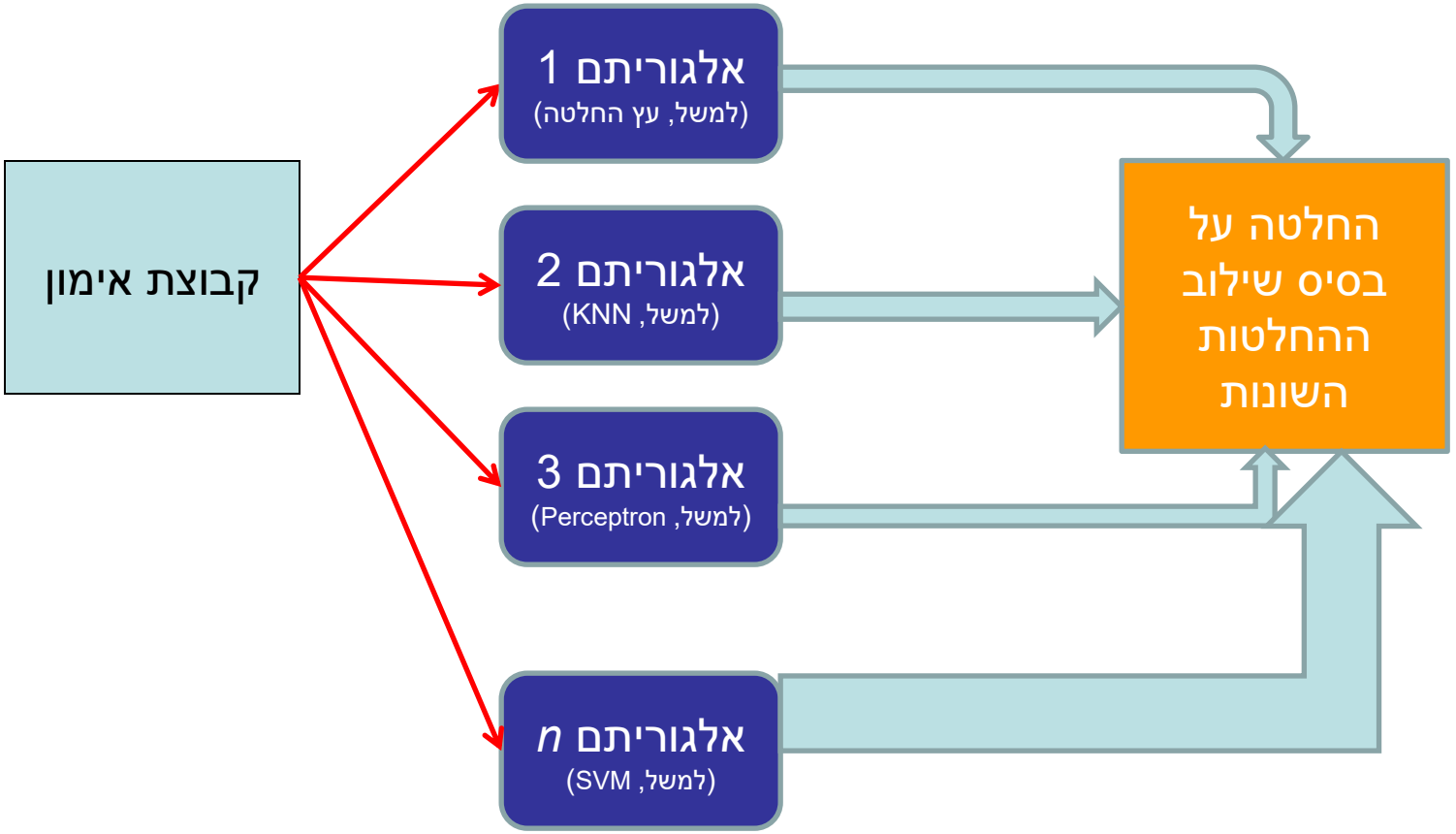


# Existing methods

- מניפולציה של **הדוגמאות**
- מניפולציה של **ה"מאפיינים**"
- מניפולציה על **ה"מחלקות**"
- הכנסת **אקראיות**

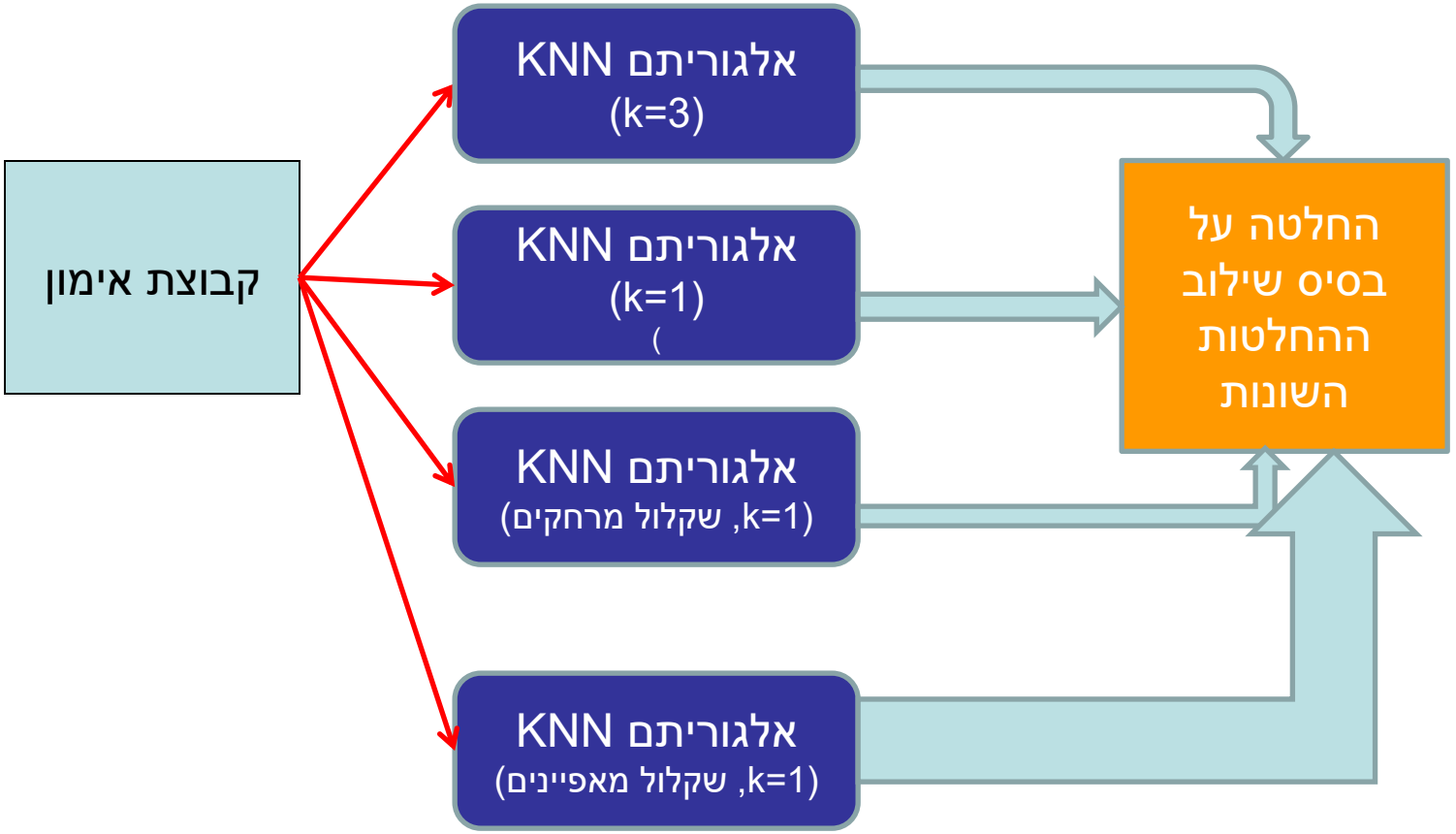
# גישות שונות ליצירת Ensemble

- על בסיס אותו Training Data הרץ אלגוריתמים שונים



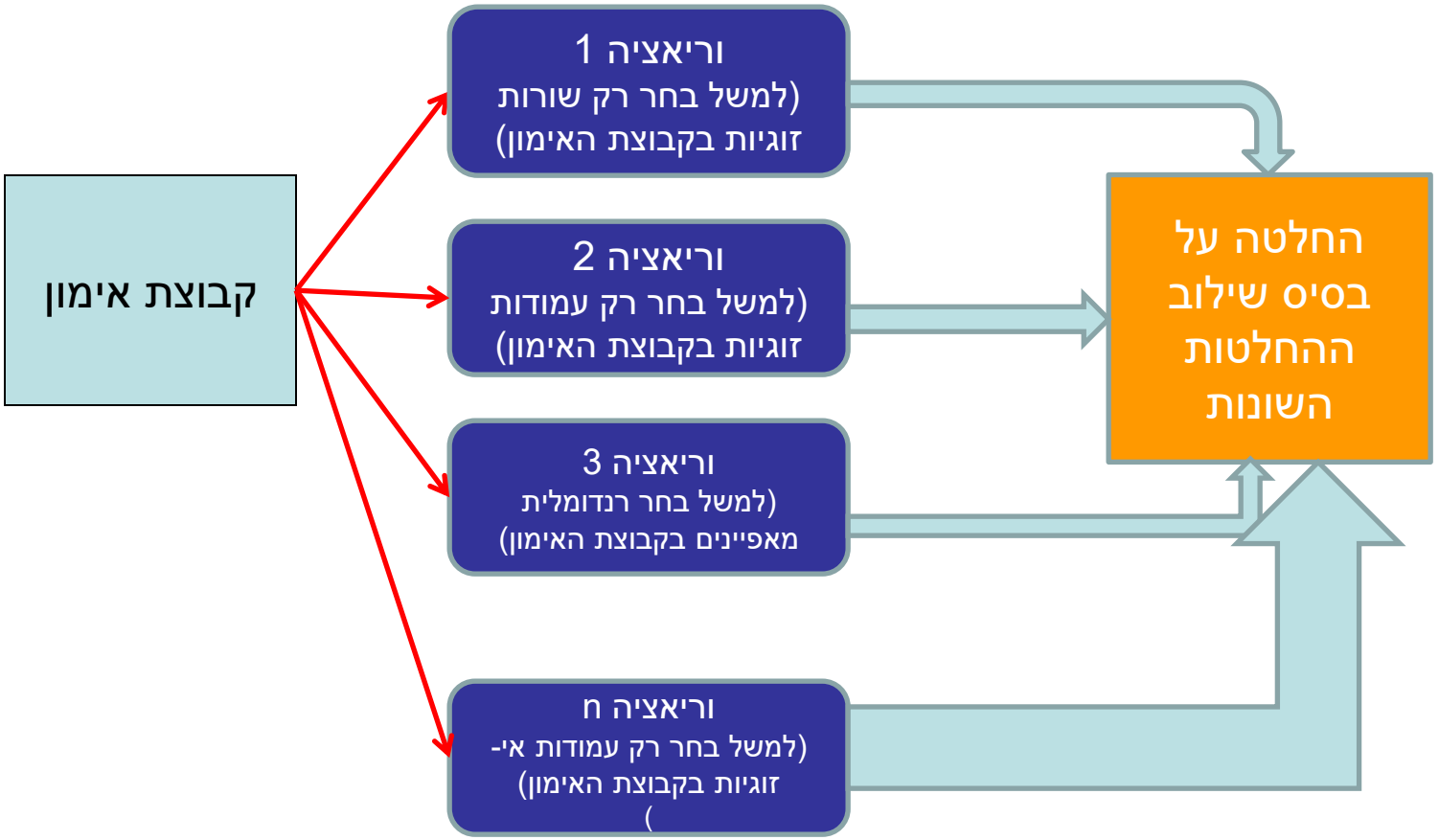
# גישות שונות ליצירת Ensemble

- על בסיס אותו Training Data הרץ את אותו אלגוריתם עם פרמטרים שונים



# גישות שונות ליצירת Ensemble

- הרץ אלגוריתמי למידה על וריאציות של קבוצת האימון



# מניפולציה של הדוגמאות

***Example method: Bagging (bootstrap aggregating)***

- הרעיון:
  - הרץ את אותו אלגוריתם למידה מס' פעמים($n$). בכל פעם בחר תת-קבוצה אחרת מקבוצת האימון המקורית. נקבל בצורה זו $n$ מסווגים שונים.

- סיווג דוגמא חדשה:
  - נריץ את n המסווגים על הדוגמא החדשה.
  - נכריע עפ"י הצבעת רוב של *n המסווגים.*

- תכונות
  - נותן תוצאות טובות עבור אלגוריתמי למידה לא יציבים

# Bagging

- אלגוריתם למידה "לא יציב"

– נאמר שמודל סיווג הינו "לא יציב" אם שינוי קטן בקבוצת האימון (למשל, הסדר שבו הדוגמאות מוזנות למערכת) יוביל להיפוטזה שונה לגמרי.

– עשוי לקרות למשל באלגוריתם רשתות נוירונים בו אנו מעדכנים עפ"י סדר הופעת הדוגמאות (מוד אינקרמנטלי)

- שיפור ביצועים באלגוריתם bagging יתקבל כאשר אין קורלציה בין המודלים השונים שנוצרו

# Bagging

- נניח שיש לנו את הדוגמאות הבאות:
  - (x1,**y1**),(x2,**y2**),(x3,**y3**)(x4,**y4**),(x5,**y5**)(x6,**y6**)(x7,**y7**),(x8,**y8**)
- נייצר מס' קבוצות אימון שונות מתוך הקבוצה הגדולה:
  - 1:(x1,**y1**),(x2,**y2**),(x3,**y3**)(x4,**y4**),(x5,**y5**)
  - 2:(x2,**y2**),(x7,**y7**),(x3,**y3**)(x1,**y1**),(x8,**y8**)
  - 3:(x1,**y1**),(x2,**y2**),(x3,**y3**)(x7,**y7**),(x5,**y5**)
  - 4:(x1,**y1**),(x6,**y6**),(x1,**y1**)(x1,**y1**),(x2,**y2**)
  - 5:(x7,**y7**),(x6,**y6**),(x1,**y1**)(x1,**y1**),(x2,**y2**)
- נייצר לכל סט "מסווג".(h1,h2,h3,h4,h5)
- בהינתן דוגמא "נעביר" דרך h1-h5 ונסווג לפי רוב

# מניפולציה של ה"מאפיינים"

## הרעיון:

מניפולציה של המאפיינים שנבחרו כקלט לאלגוריתם הלמידה, כלומר יצירת *n* מסווגים

כל מסווג נלמד על תת קבוצה שונה של מאפיינים

## סיווג דוגמא חדשה:

- נריץ את n המסווגים על הדוגמא החדשה.
- נכריע עפ"י הצבעת רוב של *n המסווגים.*

## תכונות

עובד טוב כאשר יש יתירות גבוהה בין המאפיינים

# מניפולציה על ה"מחלקות"

- באופן אקראי, חלק את K המחלקות לשתי תתי קבוצות (A,B), כלומר שייך כל אחת מהמחלקות או ל-A או ל-B
- בנה מסווג בינארי
- חזור על צעדים אלו m חזרות
- <u>סיווג דוגמא חדשה:</u>
  - סווג את הדוגמא עפ"י של אחד מ-m המסווגים שנוצרו
  - מצא את המחלקה שקיבלה את מס' ה"קולות" הרב ביותר.

# Manipulating the output targets

- Original Classes: 1,2,3,4,5,6
  - h1: A1: classes 1,2,3, B1: classes 4,5,6
  - h2: A2: classes 1,3,6 B2: classes 2,4,5
  - h3: A3: classes 2,4,6 B3: classes 1,3,5
  - h4: A4: classes 2,4,5 B4: classes 1,3,6
  - h5: A5: classes 3,4,5 B5: classes 1,2,6

- בהינתן דוגמא חדשה שסיווגה: h1 = A, h2=A, h3 = B, h4= A, h5 = A
  - מחלקה 1 מקבלת 3 "קולות" (h1,h2,h3)
  - מחלקה 2 מקבלת 2 "קולות" (h1,h4)
  - **מחלקה 3 מקבלת 4 "קולות" (h1,h2,h3,h5)**
  - מחלקה 4 מקבלת 3 "קולות" (h4,h5)
  - מחלקה 5 מקבלת 3 "קולות" (h3,h4,h5)
  - מחלקה 6 מקבלת 1 "קולות" (h2)

נסווג דוגמא זו כ-3 ←

# הכנסת אקראיות

**הרעיון**

- הכנסת אקראיות מסויימת בתהליך הלמידה.
- למשל, ישנם אלגוריתמים שבחירה ראשונית של "משקולות" עשוי להשפיע על המודל שנוצר
- באלגוריתמים אלו, הרץ את בניית המודל עם מס' "התחלות" שונות ע"מ לקבל מס' אלגוריתמים שונים

- סווג דוגמא חדשה עפ"י רוב ההצבעות.

# AdaBoost (Adaptive Boosting)

## הרעיון

- בניית מסווג לבעיית למידה ← משקל גבוה יותר לדוגמאות שסווגו באופן שגוי



- בכל שלב אנו יוצרים בעיית למידה קשה יותר
- בכל איטרציה אנו מנסים למזער את הטעות תוך שקלול משקל כל דוגמא.
- למסווג שאנו יוצרים בכל איטרציה נקרא weak classifier

***Classifying new example:***

- The final classifier is combination of the weak classifiers:

$$h_f(x) = \sum_t w_t h_t(x)$$

# AdaBoost

- given training set $(x_1, y_1), \ldots, (x_m, y_m)$
- $y_i \in \{-1, +1\}$ correct label of instance $x_i \in X$
- for $t = 1, \ldots, T$:
  - construct distribution $D_t$ on $\{1, \ldots, m\}$
  - find weak hypothesis ("rule of thumb")

    $h_t : X \to \{-1, +1\}$

    with small error $\epsilon_t$ on $D_t$:

    $\epsilon_t = \Pr_{D_t}[h_t(x_i) \neq y_i]$
- output final hypothesis $H_{\text{final}}$



From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

# AdaBoost – Finding $D_t$

- $D_1(i) = 1/m$
- given $D_t$ and $h_t$:

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} \cdot \begin{cases} e^{-\alpha_t} & \text{if } y_i = h_t(x_i) \\ e^{\alpha_t} & \text{if } y_i \neq h_t(x_i) \end{cases}$$

$$= \frac{D_t(i)}{Z_t} \cdot \exp(-\alpha_t \, y_i \, h_t(x_i))$$

where $Z_t =$ normalization constant

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \epsilon_t}{\epsilon_t} \right) > 0$$

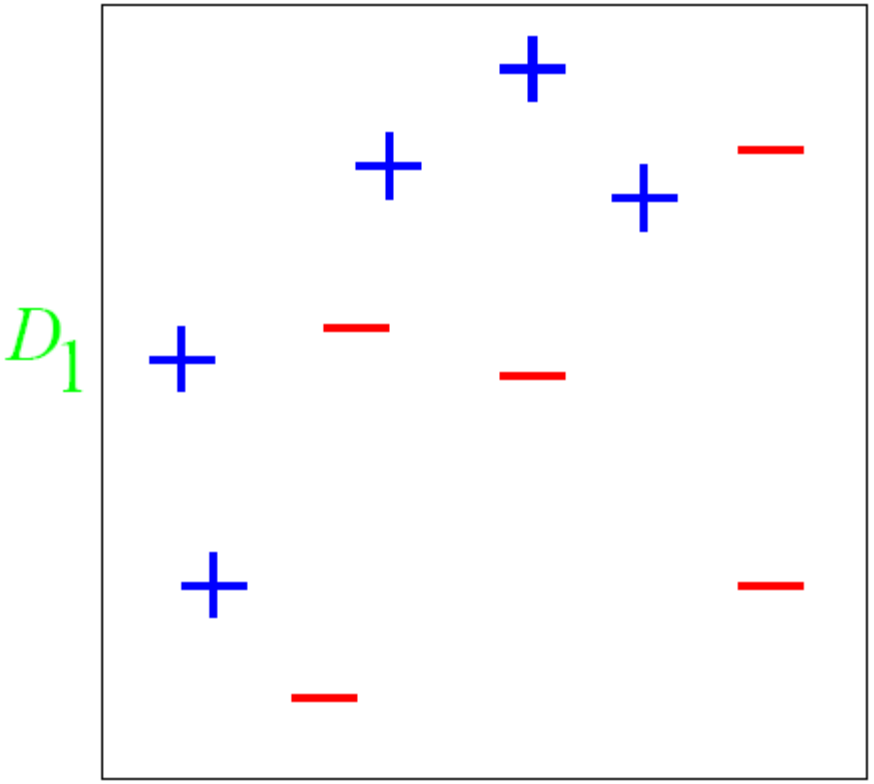


From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

# AdaBoost – Finding $H_{final}$

$$H_{final}(x) = sign(\sum_{t} \alpha_t h_t(x))$$

# Toy example

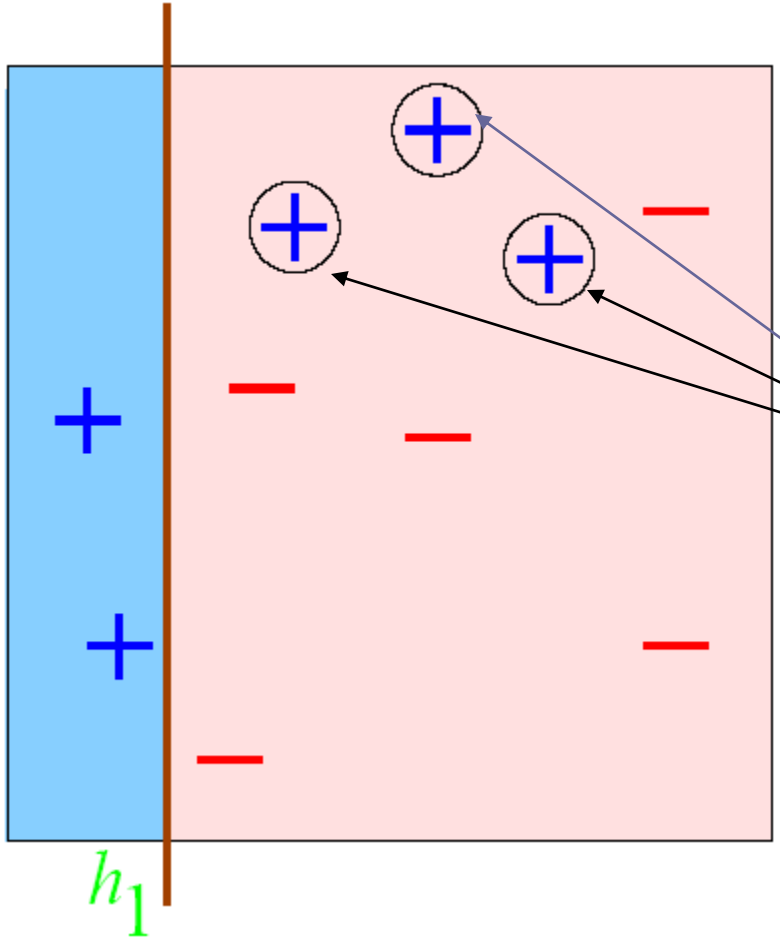



10 דוגמאות – 5 חיוביות ו-5 שליליות

אנו מתחילים שלכל הדוגמאות משקל זהה



From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

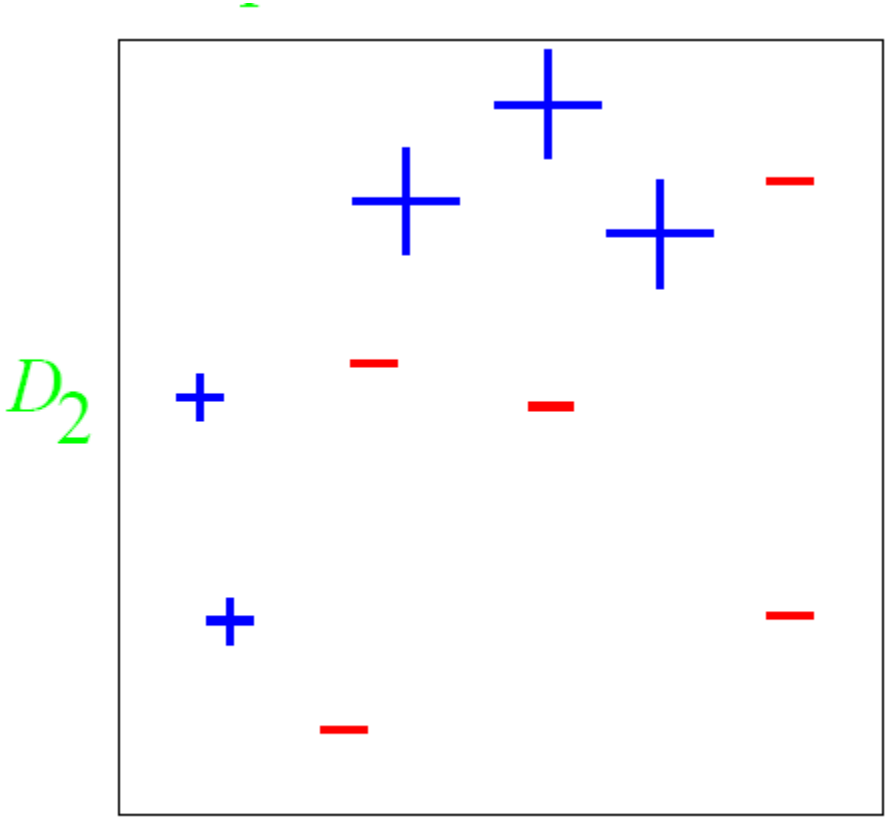



Misclassified by h1

$\varepsilon_1 = 3/10 = 0.3$



From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf




$\varepsilon_1 = 3/10 = 0.3$

$$\alpha 1 = \frac{1}{2}\ln(\frac{1-\varepsilon}{\varepsilon}) = 0.42$$

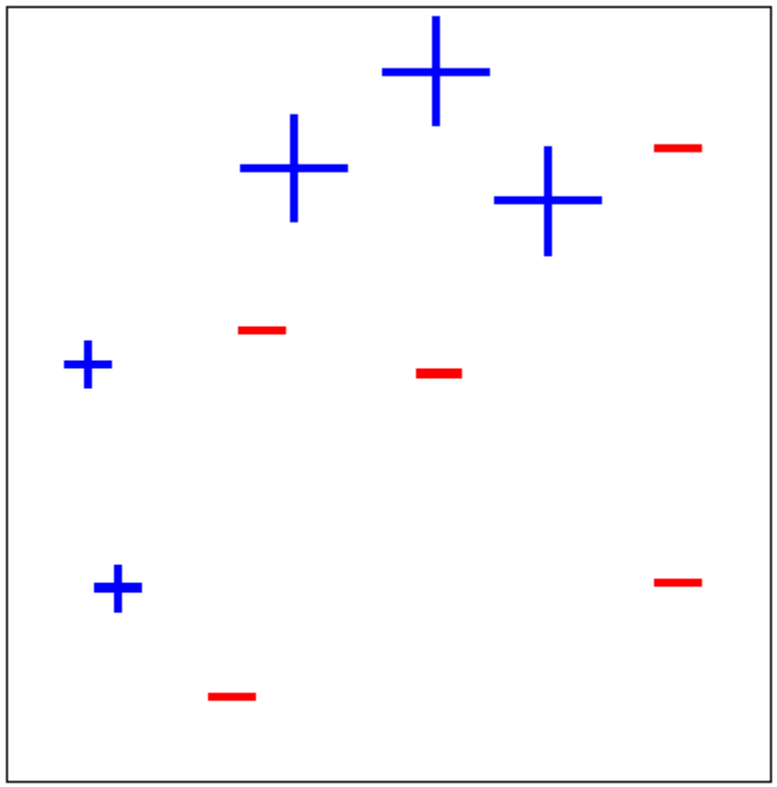


From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t}\exp(-\alpha_t y_i h_t(x_i))$$

$$\alpha_t = \frac{1}{2}\ln(\frac{1-\varepsilon_t}{\varepsilon_t})$$

# Round 2




$$D_{t+1}(correctlyClassified) = \frac{D_t(i)}{Z_t}\exp(-0.42)$$

$$= \frac{0.1}{Z_t}\exp(-0.42) = \frac{0.0657}{Z_t}$$

$$D_{t+1}(NonCorrectlyClassified) = \frac{D_t(i)}{Z_t}\exp(0.42)$$

$$= \frac{0.1}{Z_t}\exp(0.42) = \frac{0.15219}{Z_t}$$

$$7 \times \frac{0.0657}{Z_t} + 3 \times \frac{0.15219}{Z_t} = \frac{0.9164}{Z_t} \rightarrow Z_t = 0.9164$$

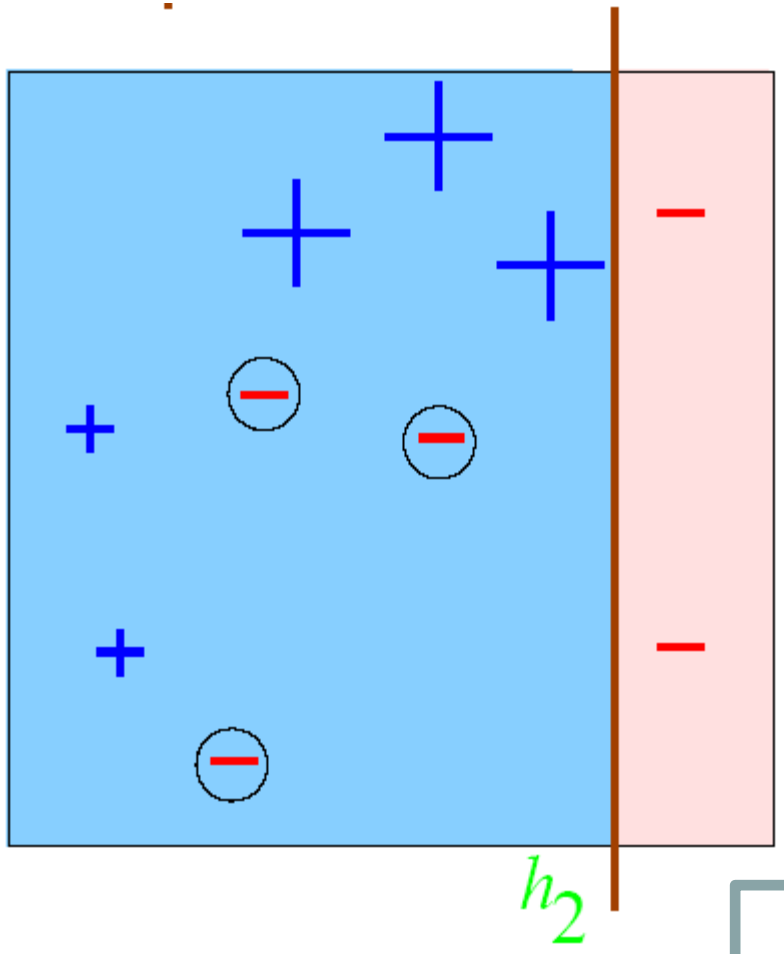


From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t}\exp(-\alpha_t y_i h_t(x_i))$$

$$\alpha_t = \frac{1}{2}\ln(\frac{1-\varepsilon_t}{\varepsilon_t})$$

# Round 2




$$D_{t+1}(correctlyClassified) = \frac{D_t(i)}{Z_t}\exp(-0.42)$$

$$= \frac{0.1}{Z_t}\exp(-0.42) = \frac{0.0657}{Z_t}$$

$$D_{t+1}(NonCorrectlyClassified) = \frac{D_t(i)}{Z_t}\exp(0.42)$$

$$= \frac{0.1}{Z_t}\exp(0.42) = \frac{0.15219}{Z_t}$$

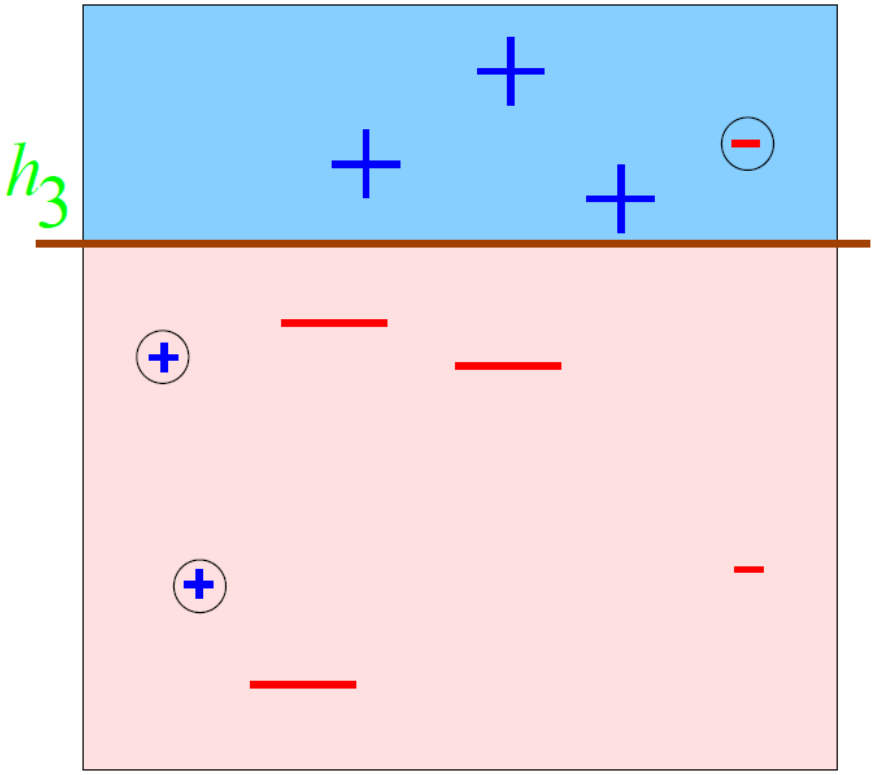
$$7\times\frac{0.0657}{Z_t} + 3\times\frac{0.15219}{Z_t} = \frac{0.9164}{Z_t} \rightarrow Z_t = 0.9164$$

$$\varepsilon_2 = 3\times\frac{0.0657}{0.9164} = 0.21$$

$$\alpha_2 = \frac{1}{2}\ln(\frac{1-0.21}{0.21}) = 0.65$$



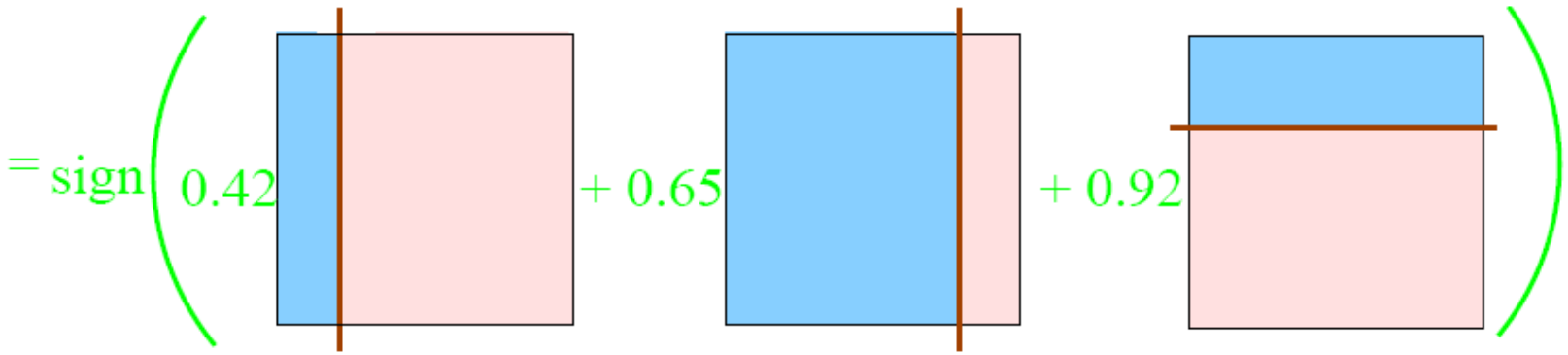
From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

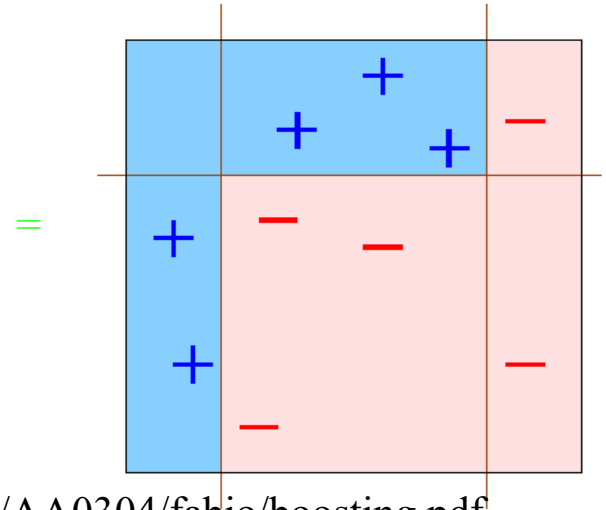



$\varepsilon_3 = 0.14$

$\alpha_3 = 0.92$

# Final classifier

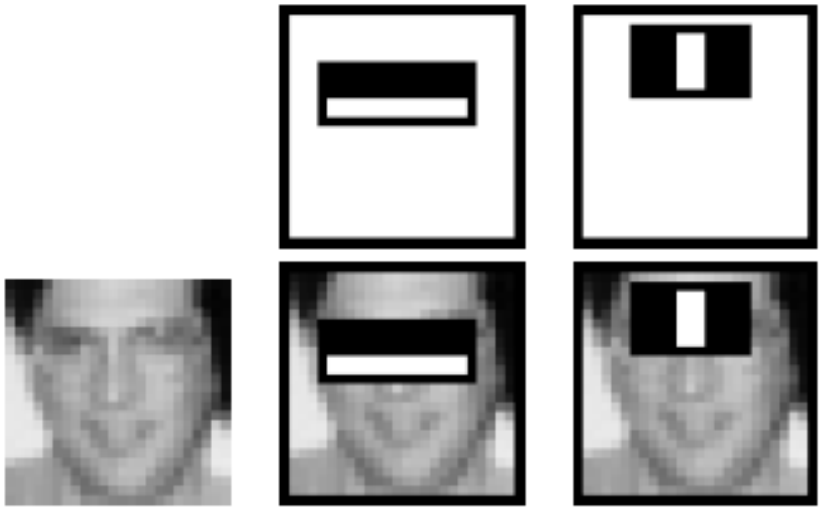



From: www.di.unipi.it/~cardillo/AA0304/fabio/boosting.pdf

# Face Detection example

- Viola&Jones system
  - **Robust Real-time Object Detection**
- Using rectangular features



*First and second used features*

# Analyzing the training error

$$\varepsilon_t = \frac{1}{2} - \gamma_t$$

Let $\forall t : \gamma_t > \gamma > 0$ → Each classifier is better than random

*Theorem:*

$$\text{Training-Error}(H_{\text{final}}) \leq e^{-2\gamma^2 T}$$

# Summary

- תהליך ה-boosting מבוסס על ההנחה שמציאת הרבה חוקים "פשוטים" (גם אם לא המוצלחים ביותר) הינו פשוט יותר מאשר מציאת אלגוריתם בודד שפותר את הבעיה.
- Adaboost
  - "רגיש" לרעשים.
  - אינו דורש ידע על המסווגים הבסיסיים מהם הוא נבנה.
  - קל ונוח למימוש
  - בהינתן שאנו יודעים שהטעות של כל "מסווג בסיס" טובה מניחוש אקראי, אזי מובטח לנו חסם של הטעות על קבוצת האימון.